

Using Google Authenticator with SSH to Secure Your Servers

This tutorial is about integrating the Google authentication mechanism in our servers to harden security. It guides readers on how to integrate Google authentication with more traditional user names/passwords and with SSH keys for better security.

Google Authenticator is a tool that generates TOTP codes, which can be used with various services. Generally, this generated code is a six-character long integer number, which is updated after every 30 seconds when installed on a smartphone (there are apps which can be used on a computer as well, but I will use the one that I have installed on my Android phone from Google Play). ‘OTP’ stands for the ‘one time password’ that is generally used with net banking, two-way authentication for Gmail and other email services, etc. ‘TOTP’ stands for ‘time-based one time password’, which means that a generated password is valid only for a specific period (30 seconds, in this case). This can also be looked upon as a two factor authentication, which can be integrated with SSH (more on this later) in order to harden the access to your server.

SSH

SSH stands for Secure Shell and, as the name suggests, it is a protocol that is used to make terminal connections with UNIX shell accounts. Although shell accounts are the least used these days, the protocol remains a standard for connections

made to servers or remote machines when a user does not have physical access to a UNIX or Linux machine. When connected, SSH provides encrypted sessions over insecure networks in a client-server architecture. SSH was introduced as a replacement for telnet and rsh, which created insecure connections. These connections are insecure because any packet capturing tool such as ‘Wireshark’ and ‘tcpdump’ can reveal the passwords, thus posing a security threat.

Various ways to secure SSH

There are basically two methods that are widely used to gain access to a server or remote machine using SSH.

user name/password: This is a very basic mechanism by which a user connects to the server with a user name and the server asks for a password. If the password is correct, access is granted; otherwise it is denied. Although SSH itself encrypts the sessions, a weak password can be cracked by brute force.

SSH public/private keys: This mechanism is more secure, as it does not need a password for a user, but a private key is needed to access the server. When a user connects to

